

Craft World External API — Scopes & Fields

Read-only partner access via OAuth 2.0 (Authorization Code + PKCE) · Base URL `/api/2/external` · Contract: `/api/2/external/openapi.json`

Access model

Scopes follow in-game visibility. Data that **every player can already observe in-game** (profile, craft world, masterpiece participation) is available to **any connected token — no scope required**; connecting the account is the consent. Owner-only data is gated behind one scope per game feature, and the user consents to each requested scope on the authorization screen. Requesting zero scopes is valid and grants the public tier only.

Public data — always included NO SCOPE

ENDPOINT	FIELDS
<code>GET /me/profile</code>	uid, displayName, avatarUrl, level, leagueId, achievements (type, subType, value, currentStreak), leagueTrophies (leagueId, rank1Count). Contains no wallet data.
<code>GET /me/craft-world</code>	experiencePoints, level, resources (symbol, amount), playerBase buildings (id, type, subType, level, pos {x, y}, upgradedAt, readyAt), ownedSpaceIds, globalCoordinate, landPlots → areas → factories (factory id/level/definition, crafting currentRunLevel/startedAt/claimedAt, boosters startTime/endTime/boostValue), mines, dynos, workers (traits, training, abilities, area lead).
<code>GET /me/masterpieces</code>	claimedMasterpieceIds, activeBattlePasses, seasonPasses, activeSeason (points, tiers with rewards, hasSeasonPass).

Private data — one scope per game feature CONSENT PER SCOPE

SCOPE	ENDPOINT	FIELDS
craft:read	<code>GET /me/craft</code>	power, powerUsed, powerLastRefill, powerMillisecondsUntilRefill, skillPoints, workshop (symbol, level), proficiencies (symbol, collectedAmount, claimedLevel), researches (symbol, claimed, remainingInMilliseconds), vaults (symbol, amount, capacity, isUnlocked, buildingUnlockLevel), currencyBalances (type, amount).
exchange:read	<code>GET /me/exchange</code>	tradeAccount (tradeCount, dailyRefillAmount, totalTradeAmount, capacity), tradeExecutions (id, errorReason, quote type/input/output, trade transactionHash/blockNumber/input/output).
onchain:read	<code>GET /me/onchain</code>	wallets (address, type, provider, providerId, primary), resourcesOnChain (symbol, amount).
inventory:read	<code>GET /me/inventory</code>	eggs (definitionId, amount), eggGuaranteeProgress, chests (definitionId, count), blueprintInventory (definitionId, amount, starLevel, landPlotUids), factoryInventory (id, definitionId, level), availableBoosters, availablePowerPacks.
purchases:read	<code>GET /me/purchases</code>	isNoAdsActive, isTransferActive, crystalPass (claimable/remaining days, hasActivePass), shopItemPurchases (shopItemId, purchasedAt), offers (items, discount, expiry), adWatchCounts.

Game data — not account-specific ANY TOKEN

ENDPOINT	FIELDS
<code>GET /game/price-list</code>	baseSymbol, prices (referenceSymbol, amount, recommendation).
<code>GET /game/dyno-production-cycle</code>	startedAt, millisecondsPerCompletion.

Good to know. Calling a private endpoint without its scope returns `403` with code `insufficient_scope`. All amounts are numbers; timestamps are ISO 8601 strings. Responses are wrapped in a `data` envelope. Requests are rate-limited per (client, user); watch the `X-RateLimit-Limit` / `X-RateLimit-Remaining` headers. A full account sync is six requests: three public endpoints plus one per granted scope.